MSc Computer Engineering, Cybersecurity and Artificial Intelligence

University of Cagliari

Department of Electrical and Electronic Engineering

Predicting Diabetes: Decision Tree vs MLP

Professors:
Ambra Demontis
Giorgio Fumera

Author:
Gianmarco Palmas (70/90/00685)

November 19, 2025

Contents

1 Introduction **1**

2 Dataset Analysis **2**

2.1 Data Description 2

2.2 Feature Scaling 3

3 Model Implementation **5**

3.1 Decision Tree Classifier 5

3.2 Multi-Layer Perceptron Classifier 8

3.3 Training tecniques 10

3.3.1 Cross Validation 11

3.3.2 Hyperparameters 12

4 Evaluation and Results **14**

4.1 Performance Metrics 14

4.2 Confusion Matrix Analysis 17

4.3 ROC Curve and AUC 18

4.4 MLP Loss Curve Analysis 19

5 Conclusion **21**

1 Introduction

This project focuses on comparing the effectiveness of two machine learning models: Decision Trees and Multi-Layer Perceptrons (MLP).

I use the Pima Indians Diabetes Dataset, which has clinical measurements from female patients, such as glucose levels, blood pressure, BMI, age, and other relevant factors. The goal is to use these traits to guess if a patient has diabetes or not.

Both models are trained and evaluated using 5-fold stratified cross-validation. This makes sure that each part of the dataset is used for testing and reduces the chance of bias in the results.

Lastly, the performance of the models will be compared using metrics like accuracy, precision, recall, F1-score, and the ROC curve, highlighting the strengths and limitations of each approach.

2 Dataset Analysis

This section provides an overview of the dataset used in the experiments. It includes a description of its structure, the features and target variable, and the overall class distribution. This provides the necessary context for the modeling and evaluation phases that follow.

2.1 Data Description

The dataset utilized in this project is the **Pima Indians Diabetes Dataset**, which consist of clinical data collected from female patients of Pima Indian descent, aged 21 years and older. It is frequently utilized as a standard dataset for binary classification tasks in healthcare machine learning.

The dataset contains 768 examples and 8 numeric features, along with one binary target variable that shows whether or not diabetes is present.

The features consist of:

- **Pregnancies**: Number of times pregnant
- **Glucose**: The amount of glucose in the plasma
- **BloodPressure**: The pressure of the bloodstream (mm Hg)
- **SkinThickness**: How thick the triceps skin fold is (mm)
- **Insulin**: Serum insulin level at 2 hours ($\mu U/ml$)
- **BMI**: Body mass index (kilograms divided by the height in meters squared)
- **DiabetesPedigreeFunction**: A function that evaluates the probability of diabetes based on familial history
- **Age**: Years of life

The outcome variable is binary: 0 for absence of diabetes and 1 for indicate diabetes.

This dataset was downloaded from Kaggle, where it is widely shared and commonly used in machine learning tutorials and experiments.

As a key part of exploratory data analysis, I generated a correlation heatmap to understand the dataset's internal structure (Figure 1). It allows us to quickly identify the linear relationships between the different variables. It helps reveal how the features relate to each other.

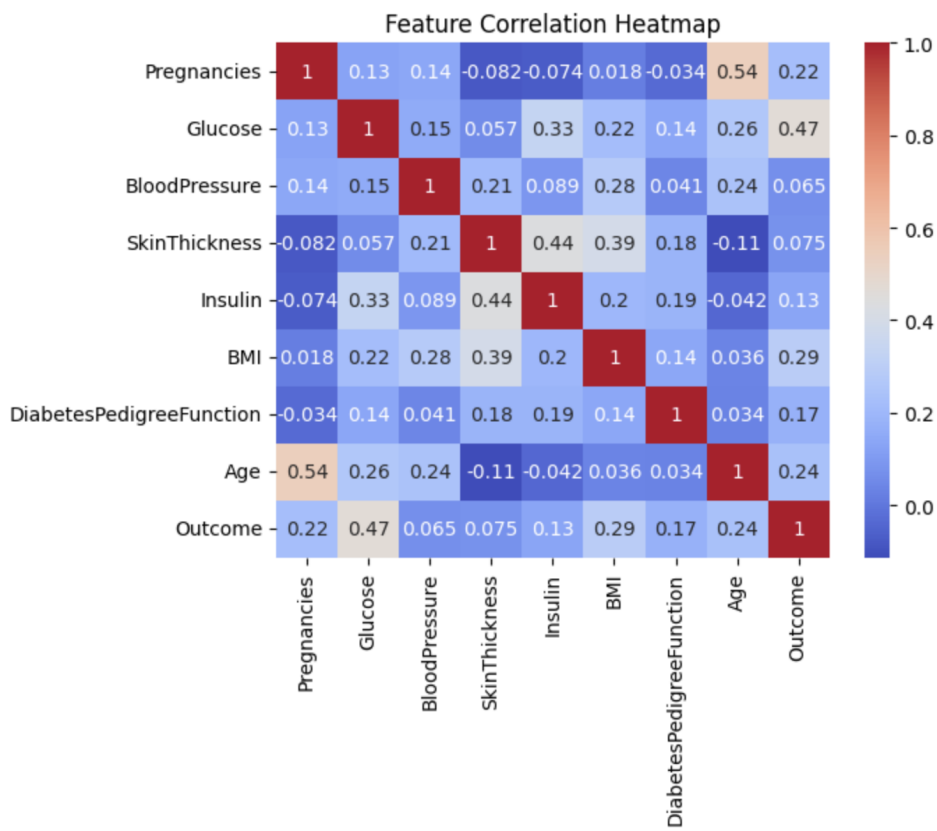

Figure 1: Correlation heatmap between the features of the Pima Indians dataset. The more intense colors indicate a stronger correlation.

2.2 Feature Scaling

Feature scaling was applied to all input variables using the `StandardScaler` provided by scikit-learn. This transformation standardizes each feature so that it has a mean of 0 and a standard deviation of 1.

This step is **not necessary for Decision Trees**. A Decision Tree only cares about finding the best threshold for a split (e.g., "is Glucose > 140?"). The actual scale of the values doesn't affect this process; it just changes the value of the threshold it finds.

However, for neural networks — such as the MLP used in this work — **scaling is crucial**. MLP models rely on gradient-based optimization. If features have very different scales (like 'Glucose' values in the 100s and 'BMI' in the 30s), the model will train slowly, become unstable, and may get stuck in a poor solution. Standardizing the features ensures all inputs are on a balanced, similar scale, which is a fundamental requirement for the MLP to train correctly and efficiently.

3 Model Implementation

This section describes the implementation of the two classification models used in the study: a Decision Tree classifier and a Multi-Layer Perceptron (MLP) neural network. Both models are trained on the same dataset using a consistent evaluation strategy to ensure a fair comparison. The following subsections outline the main characteristics of each model, as well as the training techniques and validation strategies applied during the experiments.

3.1 Decision Tree Classifier

A **Decision Tree** is a very powerful supervised machine learning model. It works like a flowchart, where each internal node corresponds to a test on an attribute (for example, the node: "level of glucose is higher than 140?"), each branch represents the test result and each leaf node represents a class label (the final decision). The path from the root of the tree to a leaf defines a specific classification rule of the form **IF condition THEN class**. This intuitive structure makes it one of the most easy-to-understand classification models, often referred to as a "white-box" model.

Structure of a Decision Tree

A decision tree is made up of several key elements:

- **Root Node**: It is the topmost node in the tree and represents the entire dataset. It is the starting point for the decision-making process.
- **Internal Nodes (or Decisional Nodes)**: These are the nodes that branch into further branches. Each internal node corresponds to a **test** on a single attribute (e.g., a condition like $x < t$) to partition the data.
- **Branches (o Arcs)**: Each branch represents a possible **outcome** of the test performed on the parent node. (e.g., if the test is $x < t$, the branches will be labeled "True" and "False").
- **Leaf Nodes (or Terminal Nodes)**: These are the final nodes at the end of each path. They don't split further and represent the final classification result. Multiple leaf nodes can predict the same class.

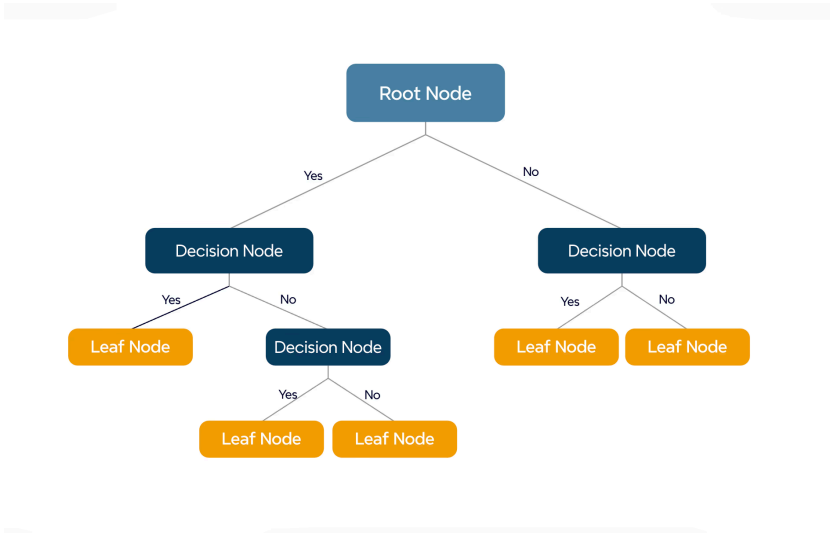

Figure 2: Generic diagram of a Decision Tree, showing the root node, internal nodes, branches and leaf nodes.

Essentially, to classify a new instance, you start at the root node and traverse the tree following the branches that correspond to the instance's feature values. This process continues until you reach a leaf node, and the class label of that leaf is assigned to the instance. The set of rules defined by all paths from the root to the leaves is **mutually exclusive and complete**, meaning that for any given instance, exactly one rule will apply.

Tree Construction and Subdivision Criteria

The main challenge in using decision trees is building the optimal tree from a training dataset. Finding the smallest and most efficient tree is a **NP-hard** problem, this means that a brute-force approach is computationally impractical. For this reason, practical learning algorithms, such as the one implemented in the `DecisionTreeClassifier` of **scikit-learn**, employ a heuristic and **"greedy"** approach, which means that they make the best decision at that moment at every single step, without worrying about the optimal long-term choice.

The algorithm builds the tree in a **top-down and recursive** manner. Starting from the root node with the entire dataset, at each step it searches for the feature and threshold that best partition the data. The "best" or "most discriminant" attribute is the one that best separates the training examples into the most **"pure"** subsets possible, that is, subsets in which most instances belong to the same class. Mathematical metrics are

used to quantify this purity.

Entropy and Information Gain

One of the most common metrics for measuring the impurities of a data set is **Entropy**, a concept derived from Information Theory. For a set of samples S with C classes, the entropy is defined as:

$$H(S) = -\sum_{i=1}^{C} p_i \log_2(p_i)$$

where p_i is the proportion of samples in S that belong to class i.

- If a set is perfectly **pure** (all samples belong to a single class), its entropy is **0** (no uncertainty).
- If a set is perfectly **impure** (samples are uniformly distributed across all classes), its entropy is at its maximum, $\log_2(C)$ (maximum uncertainty).

The goal of the algorithm is to choose the attribute for division that leads to the greatest entropy reduction. This reduction is known as **Information Gain**. At each node, the algorithm calculates the information gain for each unused attribute and selects the one with the highest gain as the splitting attribute for that node. This process is repeated recursively until a stopping condition is reached.

Overfitting and Complexity Control

A significant risk with decision trees is **overfitting**. This occurs when the tree grows too deep and complex, eventually modeling the noise in the training set rather than the underlying patterns. Such a tree perfectly classifies the training data but fails to **generalize** to new, unseen data.

To counteract overfitting, **pruning** techniques are used, which limit the growth and complexity of the tree. In this project, I employed **pre-pruning** techniques by setting hyperparameters in the `DecisionTreeClassifier`. Specifically, I chose to use the **entropy** as the splitting criterion (`criterion = 'entropy'`), which bases tree construction on the Information Gain metric described above. Other hyperparameters will be discussed in detail in a later chapter.

3.2 Multi-Layer Perceptron Classifier

The **Multi-Layer Perceptron (MLP)** is a type of Artificial Neural Network (ANN), that belongs to the class of supervised learning models. While Decision Tree creates a set of explicit rules, the MLP is inspired (in a very abstract way) by the functioning of the human brain, building a model composed of interconnected basic computational units, called artificial neurons. In this project, this model has been implemented using the class `MLPClassifier` from the `sklearn.neural_network` library.

The fundamental building block of a neural network is the **Perceptron**. A single perceptron receives a set of input, computes a weighted sum of these inputs and applies an activation function to produce an output. However, a single perceptron is a linear classifier and can only solve linearly separable problems. It notoriously fails in simple but nonlinear problems, like the logical XOR problem.

To overcome this limitation, MLP models connect multiple perceptrons (or "units") in a layered structure, allowing the network to learn and model complex and nonlinear relationships.

Feed-Forward Architecture(FF-ML)

The architecture used in this project is a *feed-forward* Multi-Layer Perceptron (FF-ML). This is the most common neural network architecture for classification and regression problems. Its main features are:

- **Layers**: Neurons are organized into sequential layers:
  1. **Input Layer**: Receives the feature values (in our case, the 8 clinical measurements).
  2. **Hidden Layers**: One or more computational layers between the input and the output. This is where the network learns complex data representations.
  3. **Output Layer**: Produces the final result (in our case, the probability of diabetes).
- **Feed-Forward**: Information moves in only one direction, from the input, through the hidden layers, to the output. There are no backward or recurrent connections.

- **Fully-Connected**: In a standard MLP, each neuron in one layer is connected to *every* neuron in the next layer.

The expressive "capacity" of the network (i.e., its ability to model complex functions) depends on the number of hidden layers and the number of neurons in each of them.

In Figure 3 it is possible to see a graphical representation of a generic MLP.

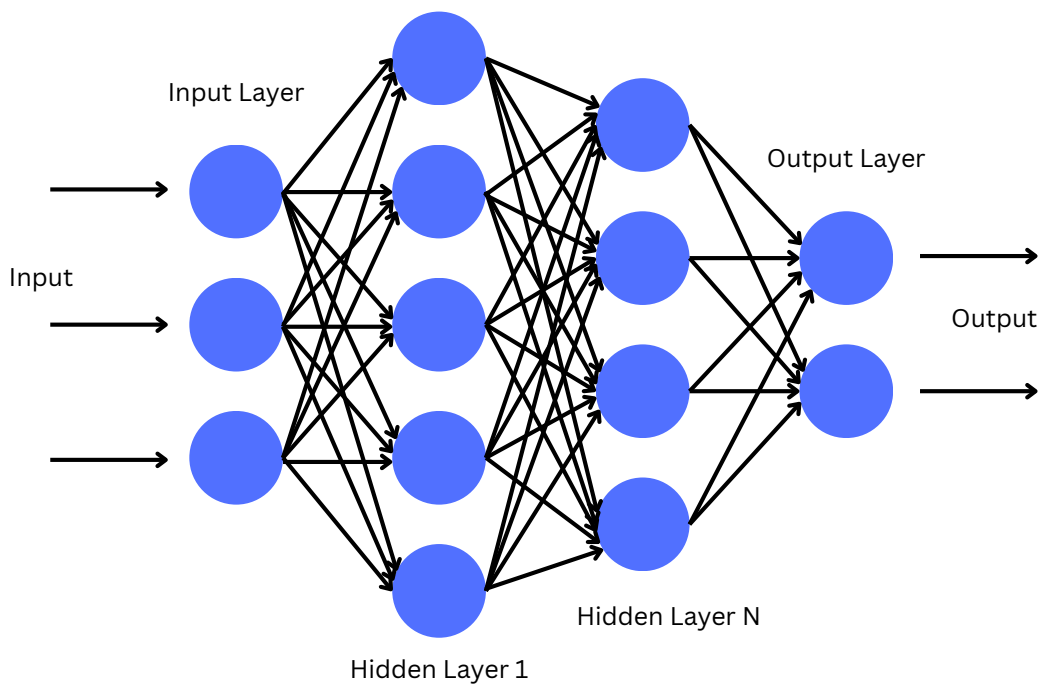

Figure 3: Generic architecture of a Multi-Layer Perceptron (MLP)

Nonlinear Activation Functions

To allow the network to learn nonlinear relationships, the output of each neuron in the hidden layers is transformed by a nonlinear **activation function**. Unlike the step function (a simple 0/1 switch) of the original perceptron, modern MLPs use continuous and differentiable functions, which are essential to the training process.

The standard choice in modern networks, as well as the one used in this project (`activation='relu'`), is the **Rectified Linear Unit (ReLU)**.

Its function is $f(x) = \max(0, x)$. ReLU is computationally very efficient and helps maintain a robust gradient flow during training, making the training process more

stable and faster, especially in networks with multiple layers.

Training using Back-propagation

The goal of training is to find the set of **weights** (the "strengths" of the connections between neurons) that minimizes the model's error on the training data. This process is managed by an algorithm called **Back-propagation**.

The algorithm runs in two phases for each training sample:

1. **Forward Propagation**: The input (e.g., the 8 features of a patient) is passed through the network, layer by layer. In each neuron, the weighted sum of the inputs is calculated and the activation function (ReLU) is applied. This process continues until the output layer, where a prediction is obtained.

2. **Error Calculation**: The prediction is compared to the actual value (the label 0 or 1) using a **Loss Function**, such as *Cross-Entropy*. The error indicates how much the network "got wrong."

3. **Backward Propagation**: This is the heart of the learning process. The algorithm calculates the error **gradient**. The gradient is a mathematical concept (a vector) that indicates the direction of maximum slope of the error function; essentially, it tells us the steepest "path" to take to reduce the error. The algorithm then calculates how much the total error is affected by each individual weight in the network, using the *chain rule* of differential calculus and propagating the error "backwards" from the output to the input.

4. **Weight Update**: The weights are updated slightly in the opposite direction of the gradient (a process called **gradient descent**) to reduce error.

3.3 Training tecniques

To ensure that the machine learning models were trained effectively and evaluated robustly, specific training techniques were employed. These methods are crucial for building models that not only perform well on the training data but, more importantly, are able to generalize to new and previously unseen data. Due to the small dataset size, I employ a **Nested Cross-Validation strategy** instead of a single Train/Test

split. This allows us to use the entire dataset for both tuning (via GridSearchCV) and evaluation (via cross_val_predict) without overfitting, ensuring statistically reliable metrics.

3.3.1 Cross Validation

A simple train-test split, in which the dataset is divided only once into a training and testing set, can lead to a performance estimate that is highly dependent on how the data is split. The result could be overly optimistic or pessimistic due to pure chance. To overcome this limitation and obtain a more reliable and stable assessment of each model's performance, **K-Fold Cross-Validation** was used in this project.

The procedure works by partitioning the entire dataset into k equally sized subsets, called "folds." The model is then trained k times. In each iteration, one of the folds is set aside as the test set, while the remaining $k-1$ folds are used for training.

Specifically, in this project I employed **Stratified K-Fold Cross-Validation** with $k = 5$. The "stratified" aspect is particularly important for datasets like the Pima Indians Diabetes, where classes may be unbalanced. Stratification ensures that each of the k folds has approximately the same percentage of samples for each target class as the full set. This prevents scenarios where a fold might, by chance, contain a disproportionately low number of diabetic or non-diabetic patients, which would lead to a biased evaluation. By using a 5-fold stratified approach, I ensure that every single data point is tested exactly once, providing a complete and less biased evaluation of the model's generalization ability.

In the code, this technique has been implemented using the scikit-learn library as follows:

```
from sklearn.model_selection import StratifiedKFold

kf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

The `kf` object as then used in two steps: first, it was provided to `GridSearchCV` (3.3.2) to find the best hyperparameters; then, it was used with `cross_val_predict`, a function that performs K-Fold to generate the predictions used to analyze the results in Chapter 4.

Figure 4: Schematic of the K-Fold Cross-Validation with $k = 5$, as used in this project.

3.3.2 Hyperparameters

Hyperparameters are configuration parameters external to the model, whose value isn't learned directly from the data but must be set before the training process begins. Their choice is crucial because they directly influence the model's complexity, its learning speed, and its ability to generalize. Careful configuration of hyperparameters is essential to find a balance between **underfitting** (a model that is too simple and fails to capture data patterns) and **overfitting** (a model that is too complex and essentially memorizes the training data, including noise).

To find the optimal values for the hyperparameters, instead of choosing them based on common practices, a systematic search was performed. For this, I used `GridSearchCV` from the `scikit-learn` library.

```
from sklearn.model_selection import GridSearchCV
```

This method performs an exhaustive ("grid") search by testing all parameter combinations specified by the user. For each combination, it uses cross-validation (in our case, the 5-fold `kf`) to calculate an evaluation score. The parameters that achieved the best average score are then selected. The hyperparameters described in the tables below are the **best parameters** found by this optimization process.

Decision Tree

Hyperparameter	Description & Justification
`criterion='entropy'`	Function to measure split quality. Chosen to use Information Gain, a classic splitting method.
`max_depth=3`	Limits the tree's maximum depth. The search identified 3 as the optimal depth, strongly favoring a simpler, more interpretable model to prevent overfitting.
`min_samples_split=5`	Min. samples required to split an internal node. The optimal value of 5 was found by the search.
`min_samples_leaf=10`	Min. samples required in a leaf node. The search selected 10, ensuring that every final decision is based on at least 10 samples, improving robustness.

Table 1: Hyperparameters for the Decision Tree model optimized via GridSearchCV.

Multi-Layer Perceptron (MLP)

Hyperparameter	Description & Justification
`hidden_layer_sizes=(32, 16)`	Sets the 2-layer architecture. The search found this larger (32, 16) structure to be optimal for balancing model capacity and risk.
`learning_rate_init=0.01`	The search identified 0.01 as the optimal value.
`alpha=0.001`	L2 regularization strength. The grid search confirmed that this small value is the optimal choice to mitigate overfitting for this model.
`activation='relu'`	Activation function for hidden layers. This was a fixed parameter; ReLU is the modern, efficient standard for stable training.
`solver='adam'`	Optimization algorithm. This was a fixed parameter; 'Adam' is a robust, fast-converging, and widely recommended default optimizer.
`max_iter=800`	Max. training epochs. A fixed high value to ensure the 'Adam' solver has sufficient time to converge.

Table 2: Hyperparameters for the MLP model optimized via GridSearchCV.

For both models, the `random_state=42` hyperparameter was set to ensure the reproducibility of the experiments, allowing anyone running the code to obtain the exact same results.

4 Evaluation and Results

The models are evaluated and their performance compared using multiple metrics. Assessment is performed through accuracy, precision, recall, F1-score, and ROC-AUC, providing a comprehensive view of the predictive capabilities. The following subsections present a detailed analysis of the results, including performance metrics, confusion matrices, and ROC curves with AUC, highlighting the strengths and limitations of each model.

4.1 Performance Metrics

To evaluate and compare the performance of Decision Tree (DT) and Multi-Layer Perceptron (MLP) models, several standard metrics for binary classification were used. Each metric offers a different perspective on the model's effectiveness. The performance of the two models can be seen in the image 5.

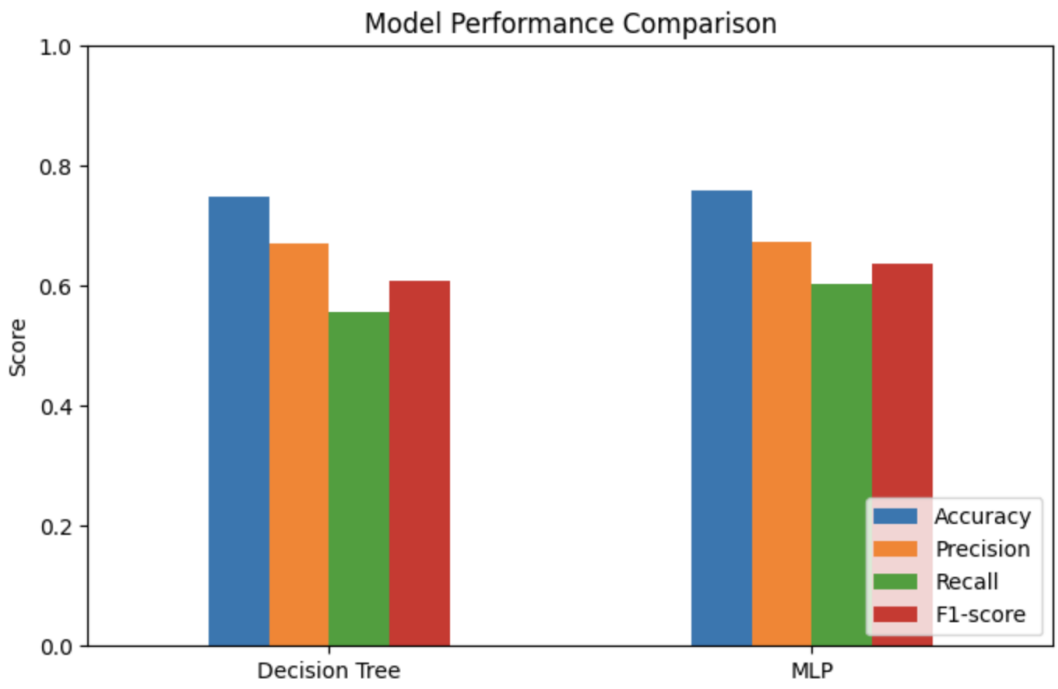

Figure 5: Performance of the models.

Accuracy

Accuracy is the most intuitive metric and represents the proportion of correct predictions (both positive and negative) compared to the total number of predictions made. It is calculated as:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

where:

- TP (True Positives): Positive cases predicted correctly.
- TN (True Negatives): Negative cases predicted correctly.
- FP (False Positives): Negative cases incorrectly predicted as positive (False Alarms).
- FN (False Negatives): Positive cases incorrectly predicted as negative (Missed Diagnoses).

Accuracy provides a general view of performance, but can be misleading with unbalanced datasets.

Precision

Precision measures the proportion of positive predictions that were actually correct. It answers the question: "Of all the patients the model identified as diabetic, how many actually had diabetes?" It is calculated as:

$$\text{Precision} = \frac{TP}{TP + FP}$$

High precision indicates that the model generates few false positives (false alarms). This is especially important when the cost of a false positive is high.

Recall (Sensitivity or True Positive Rate)

Recall measures the proportion of actual positive cases that the model was able to correctly identify. It answers the question: "Of all the patients who actually had diabetes, how many were correctly identified by the model?" It is calculated as:

$$\text{Recall} = \frac{TP}{TP + FN}$$

A high recall indicates that the model generates few false negatives (missed diagnoses). This is crucial in contexts like the medical field, where missing a positive case (e.g., a diabetic patient) can have serious consequences.

F1-Score

The F1 score is the harmonic mean of precision and recall. It provides a single score that balances both metrics. It is particularly useful when seeking a compromise between minimizing false positives and minimizing false negatives, especially in the presence of unbalanced classes. It is calculated as:

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

A high F1-Score indicates that the model performs well in terms of both precision and recall.

Analysis of Results

The results obtained for the two models are summarized in the following table:

Model	**Accuracy**	**Precision**	**Recall**	**F1-score**
Decision Tree	0.7500	0.6712	0.5560	0.6082
MLP	0.7591	0.6722	0.6045	0.6365

From the analysis of these metrics the following emerges:

- **General Accuracy**: The MLP (75.9%) shows slightly higher overall accuracy than the Decision Tree (75.0%). Both models correctly classify approximately 3/4 of patients.
- **Precision**: Both models show almost identical Precision (around 67%). This indicates that their ability to avoid "false alarms" (classifying a healthy patient as diabetic) is virtually the same.
- **Recall**: The most significant difference is seen in Recall. The MLP (60.5%) is noticeably more effective than the Decision Tree (55.6%) at identifying patients who actually have diabetes. This is a critical point, as the DT "misses" more positive diagnoses than the MLP.

- **F1-Score**: Consistently, the MLP (63.7%) achieves a better F1-Score than the Decision Tree (60.8%), indicating a superior overall balance between precision and recall, this time driven by its advantage in Recall.

In summary, based on these metrics, the MLP demonstrates superior overall performance. Its main advantage no longer lies in avoiding false positives (where the models are equivalent), but in a **markedly better ability to identify positive cases (Recall)**. This suggests that the MLP's nonlinear architecture, after optimization, was able to model more flexible and effective decision boundaries to capture the patterns of diabetic patients.

4.2 Confusion Matrix Analysis

The **Confusion Matrix** is a visual tool that compares real labels with those predicted by the model, allowing you to analyze the typology of errors made. The four cells of the matrix represent True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN), as defined in the previous section.

The confusion matrices obtained for the Decision Tree (DT) and Multi-Layer Perceptron (MLP) models are shown in Figure 6.

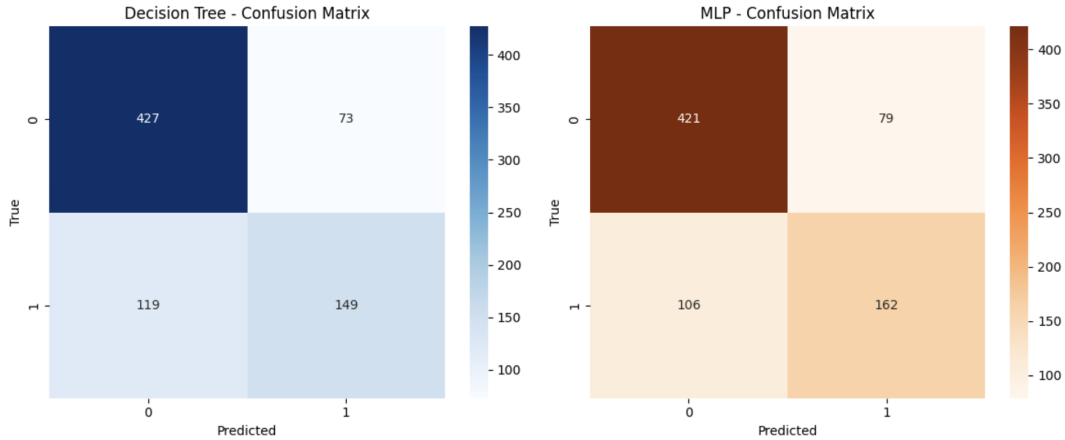

Figure 6: Confusion matrices for Decision Tree (left) and MLP (right).

A visual comparison of the two matrices highlights a clear **trade-off** in performance:

- The **Decision Tree** It handles negative cases (healthy patients) slightly better.

It obtains more True Negatives (427 vs 421) and commits fewer False Positives (73 vs 79).

- The **Multi-Layer Perceptron (MLP)** on the other hand, is markedly superior in identifying positive cases (diabetic patients). It obtains a significantly higher number of true positives (162 vs. 149) and, consequently, a much lower number of false negatives (106 vs. 119).
- This indicates that the MLP "misses" 13 fewer diagnoses than the DT, demonstrating a higher recall.

In summary, the matrix analysis shows that the MLP is more effective in finding actually sick patients (higher recall).

4.3 ROC Curve and AUC

The **Receiver Operating Characteristic** curve is a graph that shows how good a model is at distinguishing between two classes (in our case, diabetics and healthy people).

This graph correlates two metrics:

- **True Positive Rate (TPR)**: how many diabetic patients the model can **correctly identify**. (It's the Recall, $TPR = TP/(TP + FN)$).
- **False Positive Rate(FPR)**: how many healthy patients the model gets wrong, erroneously labeling them as diabetic(i **"false alarm"**). ($FPR = FP/(FP + TN)$).

A **perfect** model would be in the upper left corner: it finds 100% of diabetics (TPR=1) without committing any false alarms (FPR=0). A **useless** model follows the dotted diagonal line.

Area Under the Curve (AUC)

While the ROC curve provides a visual representation, the **Area Under the Curve (AUC)** quantifies the overall performance of the classifier into a single number. The

AUC represents the probability that the classifier will assign a higher score (rank) to a randomly chosen positive instance than to a randomly chosen negative instance.

- AUC values range from 0 to 1.
- An AUC of 1.0 represents a perfect classifier.
- An AUC of 0.5 represents a classifier with no discriminant ability, equivalent to a random prediction (the diagonal line).
- Values between 0.5 and 1.0 show how well the classifier can distinguish between classes — the closer to 1.0, the better.

Analysis of Results

The ROC curves for the Decision Tree (DT) and Multi-Layer Perceptron (MLP) models are shown in Figure 7. Looking at the graph, the MLP curve (orange line, AUC = 0.82) consistently rises above the Decision Tree curve (blue line, AUC = 0.80) for most of the range, indicating a better ability to distinguish between classes at the same false positive rate. Both models significantly outperform random classification (AUC = 0.5). The MLP's higher AUC, considered a "good" level of discrimination compared to DT's "fair/good," confirms its superior overall discriminative ability for this task, reinforcing the results seen in the previous metrics.

In summary, the ROC curve analysis reinforces the conclusions drawn from the performance metrics and confusion matrices: the MLP demonstrates better discriminative ability than the Decision Tree for this specific classification task.

4.4 MLP Loss Curve Analysis

As a final analysis, it's useful to look at the MLP model's training curve. A similar graph is not available for the Decision Tree, since its training is not an iterative, epoch-based process.

The Figure 8 shows the behavior of the cost function (Loss) of the model at each training epoch. The rapid descent of the curve, followed by a flattening, indicates that the model has learned successfully and reached convergence (i.e., a stable solution).

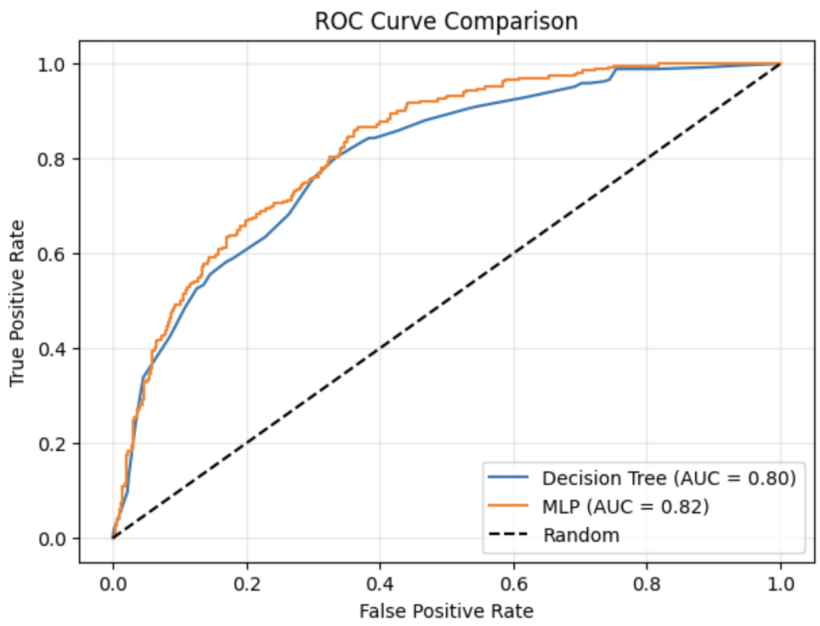

Figure 7: Comparison of ROC curves between Decision Tree and MLP models.

The graph also shows the effect of the **early stopping**: training stopped automatically after about 20 epochs (well before the 800 limit set in `max_iter`). This is an effective technique that stops training as soon as the model stops improving, thus preventing overfitting.

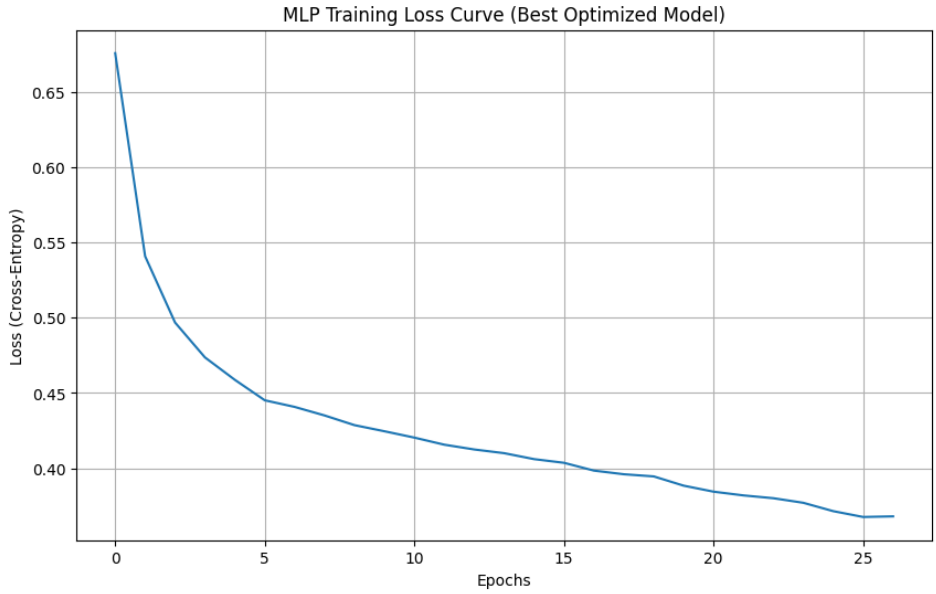

Figure 8: Training curve (Loss) of the MLP model.

5 Conclusion

This project compared a Decision Tree (DT) and a Multi-Layer Perceptron (MLP) for diabetes prediction on the Pima Indians dataset, using a 5-fold Stratified K-Fold Cross-Validation evaluation.

The results indicate that the MLP performed slightly better overall than the DT, showing an advantage across several key metrics, including overall accuracy (75.9% vs 75.0%) and area under the ROC curve (AUC = 0.82 vs. 0.80).

The analysis revealed that the MLP's advantage stems primarily from a **significantly improved ability to identify positive (diabetic) cases**. This was reflected in its higher Recall (60.5% vs. 55.6%) and its ability to find more True Positives (162 vs 149) with fewer Missed Diagnoses (106 vs 119).

However, a common **limitation of both models** emerged: difficulty correctly identifying all diabetic patients, as indicated by a suboptimal recall value (approximately 59%).

In conclusion, although the MLP offers a modest improvement in quantitative metrics, this comes at the expense of interpretability, being a "black box" model. The Decision Tree, while slightly less performant, guarantees transparency thanks to its rule-based structure. The choice between the two models therefore depends on the desired trade-off between the small performance gain and the need for model interpretability, taking into account the shared challenge of identifying all positive cases.